Lesson 6

Material

https://github.com/unixfile/learning-python-2026

Today

- Regular expressions with the `re` module
- Defining functions and arguments
- Default argument values
- Scope: where Python looks for a name
- `*args` and `**kwargs`
- Lambda expressions

1 Regular expressions

- A character sequence that defines a search pattern
- Used to search, or search-and-replace, in text
- Widely used and not unique to Python
- Powerful, but quickly get hard to read

Recommended tool

https://regex101.com

2 A first match

```
import re

anthem = "God save our gracious King"

m = re.search(r'gracious King', anthem)
m.group()   # 'gracious King'
```

- `re.search` returns a `Match` for the **first** match, or `None`
- `.group()` returns the matched text
- `r'...'` is a raw string: backslashes stay literal, so regex escapes like `\d` reach the engine unchanged

3 Common patterns

```
.          any character except newline
[abc]      any one of a, b, c
[^abc]     any character except a, b, c
\d         any digit (0-9)
\w         any word character (letter, digit, _)
\s         any whitespace (space, \t, \n)
?          0 or 1 repetition
+          1 or more repetitions
*          0 or more repetitions
^  $       start / end of string
( )        group patterns
a|b        either a or b
\.         a literal dot (escape special chars)
```

4 Greedy vs. non-greedy

```
line = "God save our gracious King"

# Greedy: longest possible match
re.search(r'G.*o', line).group()
# 'God save our gracio'

# Non-greedy: shortest possible match
re.search(r'G.*?o', line).group()
# 'Go'
```

- ?, +, * are greedy; add ? after them to go non-greedy
- Greedy matching can be expensive on large strings

5 Finding all matches

```
text = "The rain in Spain stays mainly in the plain."

# Whole words ending in 'ain'
re.findall(r'\b\w*ain\b', text)
# Output: ['rain', 'Spain', 'plain']
```

- `re.findall` returns every non-overlapping match as a list
- `\b` is a word boundary, so `mainly` is skipped (no boundary after `ain`)

6 Groups

```
text = 'Berlin 1989-11-09 22:45 UTC+1'
rgx  = r'(\d{4})-(\d{2})-(\d{2})\s(\d{2}):(\d{2})'

m = re.search(rgx, text)
m.group(0)   # '1989-11-09 22:45'
m.group(1)   # '1989'
m.group(2)   # '11'
m.group(3)   # '09'
```

- `( )` captures part of a match into a numbered group
- `group(0)` is the whole match, same as `group()`
- Named groups, flags and `re.escape`: see Further reading

7 Defining functions

```
# Definition
def multiply(a, b):
    """Return the product of two numbers."""
    return a * b

# Invocation (calling)
multiply(3, 4)     # 12
multiply(25, 25)   # 625
```

- `def` declares a function; `return` sends a value back
- A function with no `return` returns `None`

8 Arguments

```
def greet(name, age, greeting='Hello'):
    print(f'{greeting}, {name}! You are {age} years old.')

greet('Alice', 30)
# Hello, Alice! You are 30 years old.

greet(greeting='Good afternoon', age=80, name='John')
# Good afternoon, John! You are 80 years old.

greet('Taylor', 34, greeting='Hi')
# Hi, Taylor! You are 34 years old.
```

- Positional args go by order; named args make order irrelevant
- `greeting` has a default, used when not supplied
- When mixing, positional arguments must come first

9 Scope: where Python looks for a name

The same name can exist in several places. Python searches them from the inside out and uses the first one it finds:

1. **L**ocal: inside the current function
2. **E**nclosing: a function wrapped around it
3. **G**lobal: the top level of the file
4. **B**uilt-in: names Python provides, like `print`, `len`, `max`

- The first match wins; the search stops there
- This inside-out order is nicknamed the **LEGB rule**

10 Scope in practice

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)    # local
    inner()
    print(x)        # enclosing

outer()
print(x)            # global

print(max)    # <built-in function max>
max = 3       # shadows the built-in!
print(max)    # 3
```

11 The * and ** operators

```
def total(*args):       # collect into a tuple
    return sum(args)

total(1, 2, 3)          # 6

nums = [1, 2, 3]
total(*nums)            # 6  (unpack the list)

def show(**kwargs):     # collect into a dict
    print(kwargs)

show(a=1, b=2)          # {'a': 1, 'b': 2}

d = {'a': 1, 'b': 2}
show(**d)               # same, by unpacking d
```

- In a def, *args/**kwargs collect extra arguments
- In a call, */** unpack a list/dict into arguments

12 Lambda expressions

```
# These two are equivalent
def power(x, y):
    return x ** y

power = lambda x, y: x ** y

people = {'Bob': 25, 'Dave': 20, 'Alice': 30}

# Sort items by value (age)
sorted(people.items(), key=lambda item: item[1])
# [('Dave', 20), ('Bob', 25), ('Alice', 30)]
```

- A lambda is a small, anonymous, one-expression function
- Handy as a throw-away `key=` argument to `sorted`, `max`, etc.

Further reading: named groups

```
text = "Date: 2024-06-13"
rgx  = r'(?P<year>\d{4})-(?P<month>\d{2})'

m = re.search(rgx, text)
if m:
    m.group('year')     # '2024'
    m.group('month')    # '06'
```

- `(?P<name>...)` lets you reference a group by name instead of number

Further reading: flags and escaping

```
text = "April is the\ncruellest month"

# By default '.' does not match newline
re.search(r'.*', text).group()
# 'April is the'

# The DOTALL flag changes that
re.search(r'.*', text, flags=re.DOTALL).group()
# 'April is the\ncruellest month'

re.escape("$^*+?{}")
# '\\$\\^\\*\\+\\?\\{\\}'
```

- `re.escape` escapes every special character in a string

Further reading: the mutable default trap

```
def add_item(item, basket=[]):    # evaluated once!
    basket.append(item)
    return basket

add_item('a')    # ['a']
add_item('b')    # ['a', 'b']   reused!
```

```
def add_item(item, basket=None):
    if basket is None:
        basket = []
    basket.append(item)
    return basket

add_item('a')    # ['a']
add_item('b')    # ['b']
```

- A default value is evaluated **once**, when the function is defined
- Use `None` as the default and build the mutable object inside